Table of Contents

Introduction **2**

Why do we need Typescript? **3**

Convering jsx to tsx **5**

Defining tsconfig.json *5*

Changing ***`App.jsx`*** *to* ***`App.tsx`*** *6*

Change main.jsx to main.tsx *7*

Creating the `Item` *Type Definition (*`src/types/item.ts`*)* *8*

Updating `HomePage.jsx` *to* `HomePage.tsx` *8*

Change `NavMenu.jsx` *to* `NavMenu.tsx` *9*

Change ItemGrid.jsx to ItemGrid.tsx: *9*

Change ItemTable.jsx to ItemTable.tsx: *10*

Change ItemListPage.jsx to ItemListPage.tsx: *11*

Configuring environment variables **12**

Create environment files *12*

Updating Code to Use Environment Variables *13*

Benefits of Using TSX (TypeScript) **14**

Demo: Typescript

Author: Baifan Zhou
Detailed explanations are created with the help of ChatGPT/Gemini.

Introduction

This demo will transform the code we wrote in jsx into typescript.

Why Use TypeScript?

1. Static Type Checking:
2. Improved Code Quality and Maintainability:
3. Better Collaboration:
4. Modern JavaScript Features:

Compilation and Execution Process:

1. Compilation
2. Transformation
3. Bundling
4. Execution

Converting JSX to TSX:

1. **Setup**: Define tsconfig.json to ensure proper compilation of TSX files.
2. **File Conversion**: Change file extensions from .js to .tsx and add explicit type declarations.
3. **Type Checking**: Ensure type safety by checking for null values and defining types for state and props.
4. **main.tsx:** Check existence of the rootElement.
5. **App.tsx, NavMenu, HomePage:** Explicitly declare React.FC.
6. **types/Item.tsx:** define Item interface as the data model.
7. **ItemListPage**: Transform from JSX to TSX by adding types for states and props and ensuring safe API data handling.
8. **ItemTable**: Define prop types and state types explicitly.
9. **ItemGrid**: Define prop types and use Item interface for type safety.

Configuring environment variables:

1. **Centralised Configuration**: We create a .env.development file to manage the frontend port and backend API URL in one central place.
2. **TypeScript Integration**: We replace hard-coded URLs with import.meta.env.VITE_API_URL and create the vite-env.d.ts file to ensure type safety.

Why do we need Typescript?

If we look at our code

ItemListPage.jsx

```
20  const ItemListPage = () => {
21    const [showTable, setShowTable] = useState(true);
22    const [searchQuery, setSearchQuery] = useState('');
57        {showTable
58        ? <ItemTable items={filteredItems} apiUrl={API_URL} />
59        : <ItemGrid items={filteredItems} apiUrl={API_URL} />}
```

ItemTable.jsx

```
4   const ItemTable = ({ items, apiUrl }) => {
5     const [showImages, setShowImages] = useState(true);
6     const [showDescriptions, setShowDescriptions] = useState(true);
7     const toggleImages = () => setShowImages(prev => !prev);
8     const toggleDescriptions = () => setShowDescriptions(prev => !prev);
```

ItemGrid.jsx

```
3   const ItemGrid = ({ items, apiUrl }) => {
```

We could observe that two arguments (also known as parameters) are passed to the **ItemTable** or **ItemGrid** components, but no extra information is given. It's possible to make mistakes when passing arguments if the expected datatypes and structure are not clearly defined.

In addition, in the states definition of **ItemListPage**, states like items and loading are defined in a similar way, while items is an array of items, loading is a **Boolean** variable. These could lead to confusion.

In React, especially when using **TypeScript**, it's a good practice to define the expected types for properties (props) to catch these issues early.

TypeScript is a statically typed superset of **JavaScript** developed by Microsoft. It adds static type definitions to JavaScript.

JavaScript is dynamically typed, meaning types are checked at runtime. TypeScript's static typing provides compile-time type checking, reducing the risk of type-related errors. It allows developers to write code with types that are checked at compile time, providing additional tools for managing and maintaining large codebases.

Choosing TypeScript over JavaScript can provide several benefits, particularly in larger or more complex projects. To name a few:

1. Static Type Checking

- **Early Error Detection:** TypeScript catches type-related errors at compile time, which helps identify issues before runtime.

- **Type Safety:** Ensures that variables and function parameters are of the correct type, reducing the risk of type-related bugs.

2. Improved Code Quality and Maintainability

- **Clearer Contracts:** Explicit type annotations serve as documentation, making it easier to understand what data structures are expected.
- **Refactoring:** TypeScript's type system aids in refactoring code safely by ensuring that changes are consistent across the codebase.

3. Better Collaboration

- **Shared Understanding:** Types act as a contract between different parts of a codebase or between different developers, ensuring a common understanding of data structures.
- **Code Reviews:** Types make code easier to review and understand, as they explicitly define the shape and constraints of data.

4. Modern JavaScript Features

- **Transpilation:** TypeScript supports modern JavaScript features and can compile down to older JavaScript versions for broader compatibility.
- **Future-Proofing:** TypeScript incorporates features from future versions of JavaScript, allowing developers to use upcoming features early.

When to Use TypeScript:

- **Team Development:** In team environments where consistent code structure and documentation are important.
- **Large Codebases:** When working with complex or large codebases where type safety and maintainability are crucial.
- **Long-Term Projects:** When you expect the codebase to evolve over time and need robust tooling to manage changes.

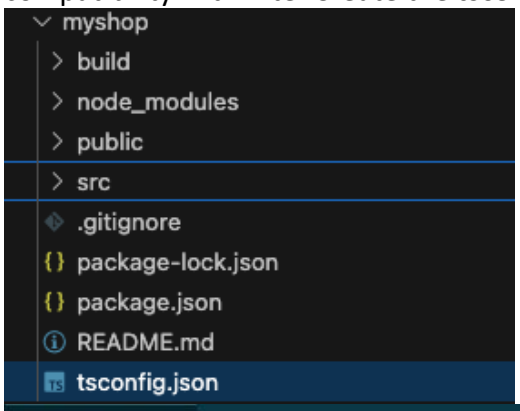
How are TypeScript and JavaScript compiled and executed?

1. **Type Checking:** TypeScript checks your code for type errors. In modern build setups (like Vite), this is usually performed by tsc in type-check mode (tsc --noEmit) during continuous integration or IDE analysis, while transpilation is handled separately by faster tools like Esbuild or SWC.
2. **Transformation & Compilation:** The build tool transforms TypeScript (.ts/.tsx) and JSX syntax into standard JavaScript. Modern React uses the **automatic JSX transform**, which automatically imports runtime functions (e.g., _jsx('div', { children: 'Hello World' })) rather than transforming code into React.createElement.
3. **Bundling:** Modern bundlers like Vite (using Rollup/Esbuild) package the transpiled JavaScript files, CSS, and assets into optimized bundles, resolving module dependencies and stripping out all type annotations.
4. **Execution:** The resulting JavaScript bundle is served to the browser or runtime environment, where it runs as standard JavaScript.

Convering jsx to tsx

Defining tsconfig.json

We start with defining a **tsconfig.json** file that ensures correct type-checking and bundling compatibility with Vite. Create the **tsconfig.json** under **myshop** and modify it.



tsconfig.json U

6-React-Intro > Demo-react-5-typescript > myshop > tsconfig.json > ...

```jsonc
{
  "compilerOptions": {
    "target": "ESNext", // Output modern JavaScript; let Vite handle browser compatibility
    "module": "ESNext", // Use native ES modules (import/export)
    "moduleResolution": "bundler", // Modern resolution mode for Vite/bundlers
    "jsx": "react-jsx", // Use the automatic JSX transform (React 17+)
    "strict": true, // Enable all strict type-checking options
    "esModuleInterop": true, // Allow default imports from CommonJS modules
    "skipLibCheck": true, // Skip type checking for library files in node_modules (faster builds)
    "forceConsistentCasingInFileNames": true, // Enforce consistent casing in file names across imports
    "noEmit": true // Only type-check — don't output .js files (Vite handles transpilation)
  },
  "include": ["src"], // Files and folders to include in the project
  "exclude": ["node_modules", "dist"] // Folders to exclude from type-checking
}
```

In a TypeScript project, it's good practice to convert `.jsx` files containing UI components to `.tsx`, and pure utility/type files to `.ts`.

Before changing the files, we first go to the folder `myshop` and run `npm install`. If the folder `node_modules` does not exist, VS Code will report many errors. That's because VS Code's built-in support for TypeScript (`.tsx`) is much stricter than its support for JavaScript (`.jsx`). The issue of missing dependencies was likely present with `.jsx` files, but it was just hidden.

How VS Code Handles JSX
For JavaScript (.jsx), VS Code's language server is very permissive. If a package like react-bootstrap isn't installed, it might not immediately show an error. Instead, it might treat the imported components as generic, undefined objects. It relies on a less strict type system and won't throw a hard error until the code is actually run and fails.

How VS Code Handles TSX
For TypeScript (`.tsx`), the language server is designed to be rigorous. When you write `import { Form } from 'react-bootstrap'`, VS Code immediately looks for the type definitions for the `react-bootstrap` library inside `node_modules` (often in a `@types`

package). If it can't find them, it correctly flags a clear error like *"Cannot find module 'react-bootstrap'"*.

Changing `App.jsx` to `App.tsx`.

Changes:

- **File extension:** Changed from .jsx to .tsx (TypeScript XML).
- **Optional:** Combining `export default` with the function declaration. This is a standard JavaScript (ES6) module syntax choice. Both ways are functionally equivalent





The return part is not changed.

Component declaration: You may see ReactFC is used in older versions of React. In the latest update, standard function syntax (`function App()`) is preferred over React.FC. The React guidelines discourage React.FC for components without explicit props, as TypeScript automatically infers the return type as JSX.Element.

You may see temporary errors while transitioning your project. That's a common TypeScript behaviour when mixing .jsx files with .tsx files. TypeScript cannot infer the types of .jsx modules out of the box, treating them as any. As we rename all remaining .jsx files to .tsx (or .ts for pure type/utility files), these errors will disappear.

Note: If VS Code fails to locate components that are already defined in .tsx files, restart the TS Server:

1. Press **Ctrl + Shift + P** (Windows/Linux) or **Cmd + Shift + P** (Mac) to open the Command Palette.
2. Type: TypeScript: Restart TS Server
3. Press **Enter**.

Component Declarations vs. Inner Helper Functions Summary
When writing React with TypeScript, we use function for **top-level components** and const for **inner helper functions**.
1. Top-Level Components: Use function
For main React components at the top of your file, standard function declarations are preferred:

- **Clean Exports:** Allows you to write export default function HomePage() {} on a single line. With a const arrow function, you can't combine export default with the declaration the same way. Instead, you'd need to declare it first and export it separately.

- **Modern React Standard:** Avoids unnecessary wrapper types like React.FC, which used to force an implicit children prop and made using generics awkward. Skipping it lets TypeScript infer the return type naturally.
- **Hoisting Flexibility:** Function declarations are hoisted, so a top-level component can be referenced earlier in the file than where it's defined. This is occasionally useful for organizing files where you want the main export near the top and supporting code below it.

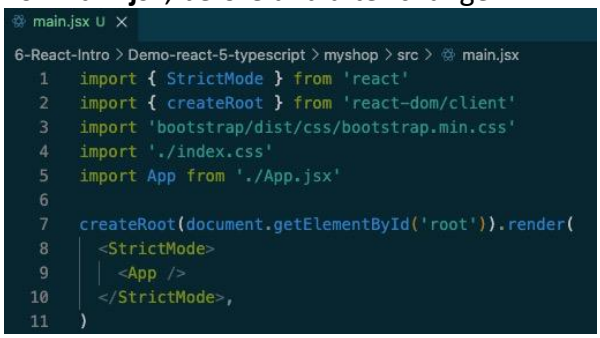
2. Inner Helper & Event Functions: Use const

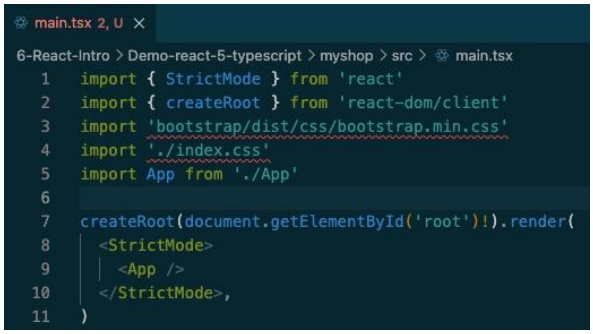
For functions defined **inside** a component (such as click handlers or state toggles), const arrow functions are preferred:

- **Prevents Accidental Reassignment:** Using const guarantees that internal logic like toggleTableOrGrid cannot be overwritten elsewhere in the component.
- **Consistent Callback Syntax:** Matches the () => {} syntax used in React event handlers and array methods like .map().
- **No Hoisting, By Design:** Unlike function declarations, `const` arrow functions are *not* hoisted: they can't be called before the line where they're defined. This is a benefit for internal helpers: it forces you to define them before you use them, keeping the logic in a component easy to read top-to-bottom instead of jumping around.

Change main.jsx to main.tsx

For **main.jsx**, before and after change:





In standard JavaScript, `document.getElementById('root')` is passed directly. However, in TypeScript, `document.getElementById('root')` returns `HTMLElement | null`. TypeScript will flag a type error if `null` is passed to `createRoot`.

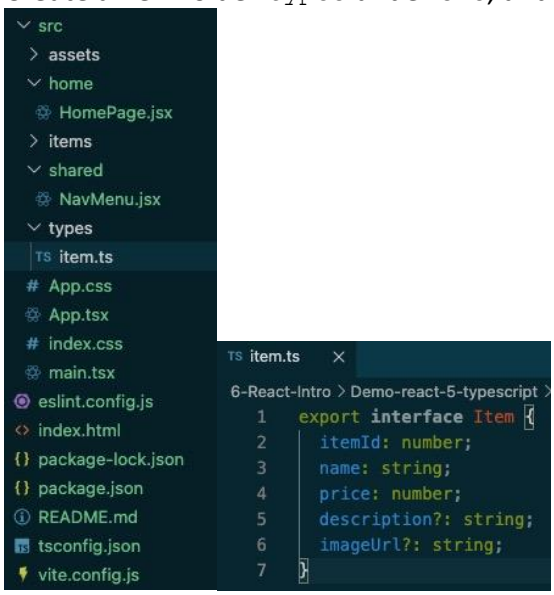
- **File Extension:** Changed from main.jsx to main.tsx.
- **Import update:** Removed the .jsx extension from the ./App import so TypeScript resolves it automatically.
- **Non-Null Assertion (!):** The exclamation mark ! tells TypeScript that document.getElementById('root') is guaranteed to exist in the HTML DOM and will not be null.
- **<StrictMode>:** Retained to catch common bugs early during development. Read more: https://react.dev/reference/react/StrictMode

Type Checking: document.getElementById('root') in TypeScript returns a value of type HTMLElement | null. TypeScript enforces checking that rootElement is not null before proceeding, ensuring that ReactDOM.createRoot is not called with null.

Conditional Check: The if (rootElement) check ensures that the code only attempts to create the root and render the React component if rootElement is not null. This prevents potential runtime errors when the element with the ID 'root' is not present in the HTML.

Creating the `Item` Type Definition (`src/types/item.ts`)

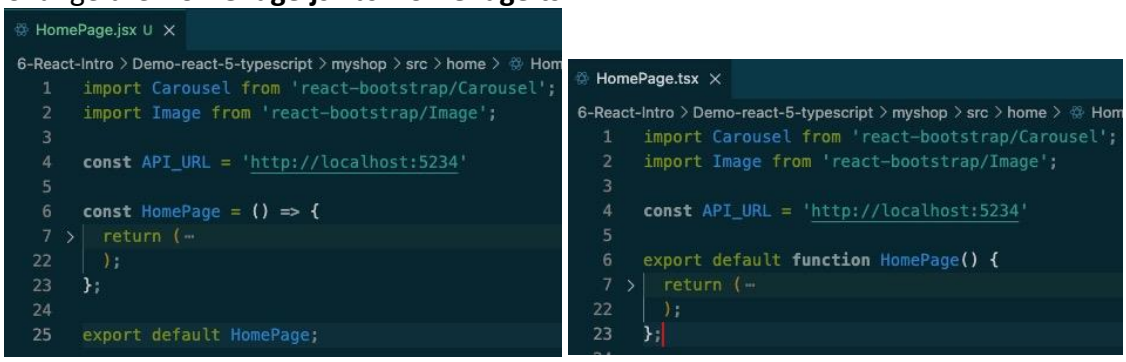
Create a new folder `types` under `src`, and create a file named `item.ts` inside it:



- **File Extension (.ts vs .tsx):** Use .ts instead of .tsx because this file only contains TypeScript interfaces and no JSX elements.
- **Interface:** The Item interface serves as our data model (similar to a class or struct in C#) and will be imported across all item-related modules for strict type checking.

Updating `HomePage.jsx` to `HomePage.tsx`

Change the **HomePage.jsx** to **HomePage.tsx**:

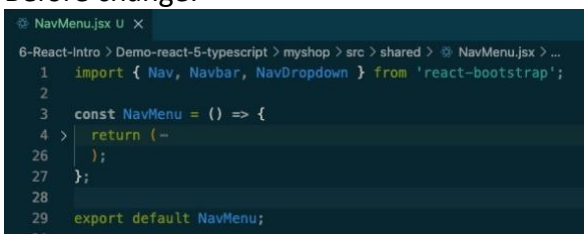


- **File Extension:** Rename file extension from .jsx to .tsx.
- **Component Function:** Standard export function syntax is used. TypeScript automatically infers the return type as JSX.Element without requiring explicit annotations since this component takes no props.

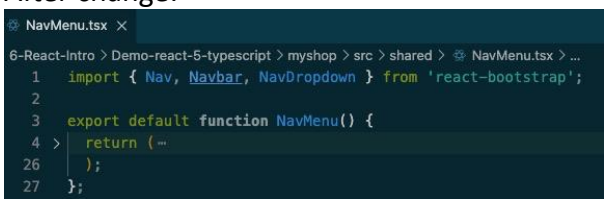
Change `NavMenu.jsx` to `NavMenu.tsx`

- **File Extension:** Changed from NavMenu.jsx to NavMenu.tsx.
- **Component Function:** Used standard function syntax for type inference.

Before change:

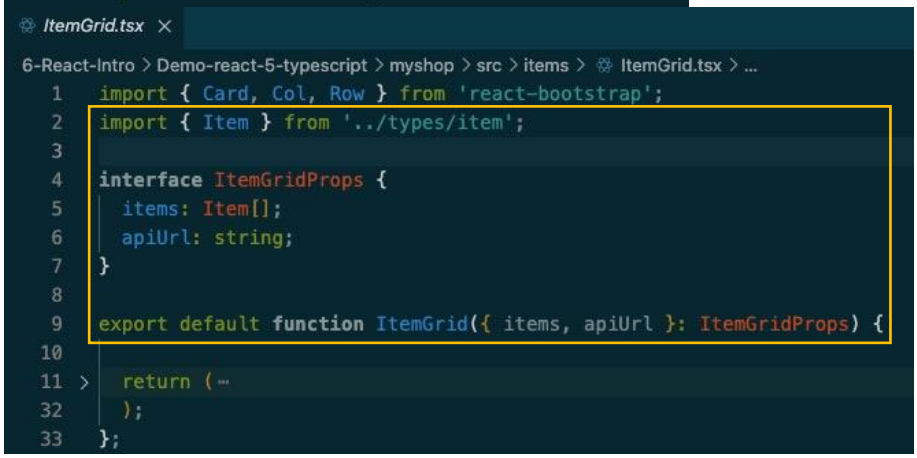


After change:



Change ItemGrid.jsx to ItemGrid.tsx:





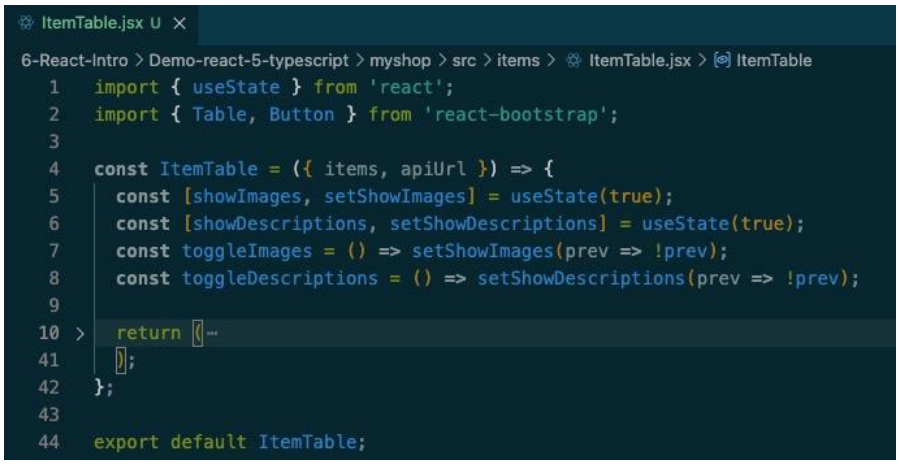
Key Changes:

- **Line 2:** Imports the Item type interface from the types/item file.
- **Lines 4–7:** Defines an ItemGridProps interface to explicitly define the expected props, declaring items as an array of Item objects (Item[]) and apiUrl as a string.

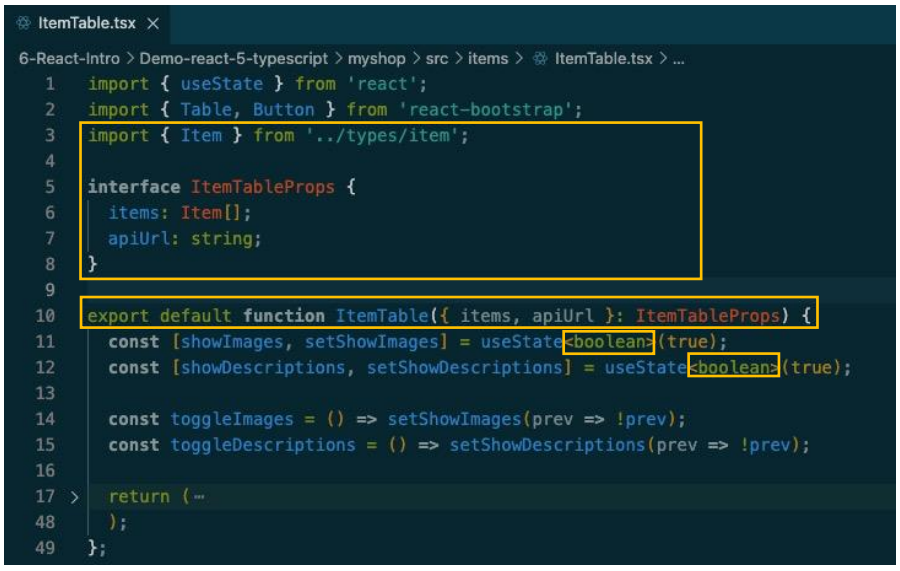
- **Line 9:** Annotates the component parameters directly ({ items, apiUrl }: ItemGridProps). This approach provides clean type checking.
- **return JSX Body:** Remains identical, but TypeScript now automatically validates every property accessed on item (e.g., item.itemId, item.imageUrl, item.name, item.price) against the Item interface.

Change ItemTable.jsx to ItemTable.tsx:

Before:



After:



- **Props Interface:** The ItemTableProps interface defines explicit types for items (Item[]) and apiUrl (string).
- **Direct Parameter Annotations:** Destructured props are typed directly (({ items, apiUrl }: ItemTableProps)), avoiding React.FC in alignment with modern React guidelines.
- **State Typing:** The useState hooks leverage generic syntax (useState<boolean>(true)), explicitly enforcing boolean state values.
- **Type Inference:** Handler functions (toggleImages, toggleDescriptions) omit explicit return annotations, as TypeScript automatically infers their return type as void.

Change ItemListPage.jsx to ItemListPage.tsx:

After:



Changes:

- **Item import:** Imports the Item type interface from the types/item file.
- **items state:** React Query owns this state via the data field of useQuery, defaulted to [] and typed as Item[] through the useQuery<Item[], Error> generic.
- **loading state:** React Query provides isLoading: boolean directly, no separate useState<boolean> needed.
- **error state:** Owned by React Query. Passing Error as the second generic argument to useQuery<Item[], Error> types error as Error | null, rather than manually managing a string | null state variable.
- **showTable state:** Still a plain useState<boolean>. This is local UI state React Query has no involvement in.
- **searchQuery state:** Still a plain useState<string> for the same reason.

- **fetchItems function:** A `Promise` represents a value that will be available at some point in the future, once an asynchronous operation completes. Since `fetchItems` is an `async` function that eventually returns an array of items, its type is `Promise<Item[]>` rather than `Promise<void>`. This return type also flows into `useQuery`'s type inference, so `useQuery` knows what shape of data it's working with.
- **Type assertion → type annotation:** When we write `const data: Item[] = await response.json()`, we are annotating the variable, telling TypeScript up front what type `data` should be. This differs from a type assertion (`as Item[]`), which simply overrides TypeScript's own checking and says "trust me." An annotation gives TypeScript a real type to check against, so if the assigned value clearly conflicts with `Item[]`, TypeScript will still catch it.
- `item.description ?? ''` falls back to an empty string when `description` is `undefined`, so `.toLowerCase()` always has a real string to work with. An item with no description just won't match on that field.

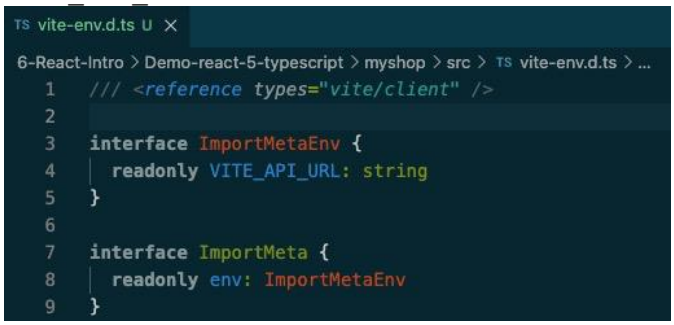
Configuring environment variables

Create environment files

Create a new file .env.development under the myshop root directory. In this file, we configure the API URL for the backend service.
Note: Vite requires custom environment variables to start with the VITE_ prefix to expose them to client-side React code.
1. Create myshop/.env.development Add the backend server URL (http://localhost:5234). This must match the applicationUrl defined in your backend's api/Properties/launchSettings.json.



2. Create `myshop/src/vite-env.d.ts` Add the TypeScript type definitions for `import.meta.env`. Make sure to include the `/// <reference types="vite/client" />` line at the very top so TypeScript recognises Vite's built-in types alongside your custom `VITE_API_URL` interface.



Updating Code to Use Environment Variables

1. Replace Hardcoded API URLs in Components In both ItemListPage.tsx and HomePage.tsx, replace const API_URL = 'http://localhost:5234' with: const API_URL = import.meta.env.VITE_API_URL;

2. Configure the Frontend Server Port in Vite Open vite.config.js (or vite.config.ts) and add the server block with port: 4000.

- **Why port 4000?** Vite defaults to port 5173. Setting the port directly in vite.config ensures our frontend consistently runs on port 4000. You can also change to other ports as long as it does not conflict with the .NET backend running on port 5234.

Benefits of Using TSX (TypeScript)

In standard **JSX**, if you forget to import a component (such as <Carousel/> in HomePage.jsx), the error won't be caught until you run eslint, or run the application and view the broken page or check the browser console.

HomePage.jsx U

6-React-Intro > Demo-react-4-2-conditionalrendering > myshop > src > home > HomePage.jsx > ...

```jsx
// import Carousel from 'react-bootstrap/Carousel';
import Image from 'react-bootstrap/Image';

const API_URL = 'http://localhost:5234'

const HomePage = () => {
  return (
    <div className="text-center">
      <h1 className="display-4">Welcome to My Shop</h1>
      <Carousel>
        <Carousel.Item>
          <Image src={`${API_URL}/images/pizza.jpg`} className="d-block w-100" alt="Pizza" />
        </Carousel.Item>
        <Carousel.Item>
          <Image src={`${API_URL}/images/fishandchips.jpg`} className="d-block w-100" alt="Fish and Chips" />
        </Carousel.Item>
        <Carousel.Item>
          <Image src={`${API_URL}/images/tacos.jpg`} className="d-block w-100" alt="Tacos" />
        </Carousel.Item>
      </Carousel>
    </div>
  );
};

export default HomePage;
```

In **TSX**, TypeScript performs static analysis directly in your editor. If you forget an import in HomePage.tsx, your code editor will immediately show a red squiggle underline and report a compile-time error before you even open the browser.

HomePage.tsx 8

6-React-Intro > Demo-react-5-typescript > myshop > src > home > HomePage.tsx > ...

```tsx
// import Carousel from 'react-bootstrap/Carousel';
import Image from 'react-bootstrap/Image';

const API_URL = import.meta.env.VITE_API_URL;

export default function HomePage() {
  return (
    <div className="text-center">
      <h1 className="display-4">Welcome to My Shop</h1>
      <Carousel>
        <Carousel.Item>
          <Image src={`${API_URL}/images/pizza.jpg`} className="d-block w-100" alt="Pizza" />
        </Carousel.Item>
        <Carousel.Item>
          <Image src={`${API_URL}/images/fishandchips.jpg`} className="d-block w-100" alt="Fish and Chips" />
        </Carousel.Item>
        <Carousel.Item>
          <Image src={`${API_URL}/images/tacos.jpg`} className="d-block w-100" alt="Tacos" />
        </Carousel.Item>
      </Carousel>
    </div>
  );
};
```

While ESLint can catch missing imports or unused variables in standard .jsx files, TSX provides deep type safety that linters cannot check:

- **Prop Type Safety & Autocomplete:** In .jsx, ESLint won't warn you if you pass a string to a component expecting a number, misspelled a prop name (e.g., itemTitle instead of title), or forgot a required prop. In .tsx, your code editor provides instant red squiggly underlines and auto-completes prop names as you type.
- **Catching Backend Data Mismatches:** When fetching data from VITE_API_URL, ESLint cannot know what properties exist on the returned API object. In .tsx, defining an interface for your API data ensures that if you try to access item.pricee instead of item.price, TypeScript flags the typo immediately before you ever open the browser.
- **Safe Refactoring:** If you rename a prop or data model field, TSX flags every file across your entire application that broke because of the change. ESLint alone cannot track data dependencies across components like TypeScript does.

Run the Project

In your terminal, navigate to the myshop directory and run:

- `npm install`
- `npm run dev`

Your app will launch at http://localhost:4000 and pull data from your backend running at http://localhost:5234. You will see the exact same UI results as before, but with type safety and environment-driven configuration.

Congratulations! You have learnt how to use TypeScript for a clearer and more maintainable project! ☺